Federated Learning 実験コード 利用マニュアル

Adaptive Model Poisoning Attack と Robust Aggregation の評価環境

2026 年 8 月（更新版）

この文書について

本書は「実験計画書」（Adaptive Model Poisoning Attack と Robust Aggregation の評価に関する実験計画書）の 7.1〜7.7 で実装したコード一式について、**何を実装したか・パラメータの変更方法・ログ/出力の読み方**をまとめたものである。コード自体の実装意図は各ファイル冒頭の docstring にも記載してあるので、併せて参照すること。本版はコード一式との突き合わせにより内容を検証し、単体テストの torch 依存に関する記述を修正、ファイル構成表に未記載だったスクリプトを追記した。

目次

1 全体像 2

1.1 何を作ったか 2

1.2 ファイル構成 3

2 クイックスタート 4

2.1 1 回だけ実験を実行する 4

2.2 パラメータを一時的に変えて実行する 4

2.3 単体テストを実行する 4

3 パラメータの変更方法 4

3.1 基本の学習設定 5

3.2 Robust Aggregation 関連 5

3.3 データ分布（IID / Non-IID） 5

3.4 悪性クライアント生成 6

3.5 攻撃設定 6

3.6 出力先 6

4 Robust Aggregation の仕組み 6

5 攻撃の仕組み 7

5.1 sign_flip / scale（更新量ベース） 7

5.2 adaptive（Adaptive Model Poisoning Attack） 7

6 run_experiments.py の使い方 7

6.1 仕組み 7

6.2 完了判定と resume（再開） 7

6.3 失敗の記録 8

7 ログ・出力ファイルの読み方 8
7.1 logs/results.csv（ラウンドごとの結果） 8
7.2 logs/summary.csv（実行1回につき1行） 8
7.3 logs/plots/（visualize_results.py の出力） 8
7.4 その他の個人ログ（笠原さん用、提出物ではない） 9
8 よくある注意点・トラブルシューティング 9
9 実験計画書との対応表 10

1 全体像

1.1 何を作ったか

Flower（PyTorch 連携）をベースにした Federated Learning 環境に、以下を追加実装した。

- **Robust Aggregation**：FedAvg / Median / Trimmed Mean / Krum / Bulyan の 5 手法
- **悪性クライアント生成**：割合 (malicious-ratio) からの seed 固定自動生成、または手動指定
- **攻撃**：sign_flip / scale（更新量ベース）、Adaptive Model Poisoning Attack（Fang ら [1] の考え方をベースにした独自実装）
- **実験の自動化**：全条件（集約手法 × 悪性割合 × データ分布 × seed）を自動で回す `run_experiments.py`
- **ログ・可視化**：条件つき CSV 保存と、比較図を自動生成する `visualize_results.py`

1.2 ファイル構成

ファイル	役割
`pyproject.toml`	実験パラメータの既定値を定義する設定ファイル（§3 参照）
`aggregation.py`	Robust Aggregation の中核ロジック（FedAvg/Median/Trimmed Mean/Krum/Bulyan）。numpy のみに依存し、Flower に依存しない純粋関数として実装
`malicious.py`	悪性クライアント ID の決定ロジック（比率からの自動生成／手動指定）
`attack.py`	攻撃の実装。sign_flip/scale（クライアント側で適用）と Adaptive Model Poisoning Attack（サーバ側で適用）
`pytorchexample/robust_strategy.py`	上記 aggregation.py・attack.py を Flower の Strategy に接続する「配線」部分（RobustFedAvg クラス）
`pytorchexample/server_app.py`	サーバ側のエントリポイント。pyproject.toml の設定を読み込み、Strategy を組み立てて学習を開始する
`pytorchexample/client_app.py`	クライアント側のエントリポイント。ローカル学習・攻撃の適用（sign_flip/scale）を行う
`pytorchexample/task.py`	モデル定義（全結合 DNN）・データ分割（IID/Non-IID）・学習/評価ループ
`pytorchexample/log_output.py`	CSV ログ出力の実体（§7 参照）
`run_experiments.py`	全実験条件を自動で回すオーケストレーションスクリプト（§6）
`visualize_results.py`	実験結果から比較図を生成するスクリプト（§7.3）
`test_aggregation.py` `test_malicious.py`	対応するモジュールの単体テスト。**numpy ダミーデータのみで完結し、torch 非依存**（§2.3 参照）
`test_attack.py`	attack.py の単体テスト。`adaptive_attack`（numpy 純粋関数）は torch 非依存だが、`sign_flip_delta` / `scale_delta` を実際の torch テンソルで検証する箇所があるため、**このファイルのみ torch が必要**（§2.3 参照）
`analyze_logs.py`	初期の動作確認用に作った旧スクリプト（*`logs/server_metrics.csv`* の accuracy/loss を *`plt.show()`* で表示するだけ）。現在は *`visualize_results.py`*（§7.3）と *`log_output.py`* の *`plot_all()`* が同等以上の機能を持つため実質的に不要。他のモジュールから import もされていない。削除しても差し支えないが、参考として残置している

メモ

`aggregation.py` ・ `malicious.py` ・ `attack.py` はいずれも「numpy の純粋関数として実装し、Flower/torch への依存を最小限にする」という設計方針で統一している。これにより単体テストが軽量になり、集約・攻撃ロジックだけを独立して検証できる（ただし `test_attack.py` は sign_flip/scale の検証用に torch を使う点に注意。詳細は §2.3）。

2 クイックスタート

2.1 1 回だけ実験を実行する

```
source .venv/bin/activate
flwr run . local-simulation --stream
```

pyproject.toml に書かれた既定値でそのまま 1 回実行される。`--stream` を付けないと、実行が非同期で投入されるだけで即座にコマンドが返ってしまう（完了を待たない）ので注意すること（§8 参照）。

2.2 パラメータを一時的に変えて実行する

```
flwr run . local-simulation --stream --run-config \
  'aggregation-method="krum" malicious-ratio=0.2 data-split="non_iid" num-server-rounds=10'
```

`--run-config` で指定したキーだけが pyproject.toml の既定値を上書きする。ただし、上書きしたいキーは事前に pyproject.toml の `[tool.flwr.app.config]` に存在している必要がある（存在しないキーを指定すると `Invalid run configuration` エラーになる）。

2.3 単体テストを実行する

```
python3 test_aggregation.py
python3 test_malicious.py
python3 test_attack.py
```

`test_aggregation.py` と `test_malicious.py` は torch 非依存・numpy ダミーデータのみで完結するため数秒で終わる。

> **修正** `test_attack.py` は `adaptive_attack`（numpy純粋関数）のテストに加え、`sign_flip_delta` / `scale_delta` を実物の `torch.Tensor` の辞書で検証する箇所があるため（test_attack.py 冒頭で `import torch`）、実行には torch のインストールが必要である。「3ファイルともtorch不要」という記述は誤りだったため訂正した。もっとも本プロジェクトは torch 自体が pyproject.toml の必須依存関係なので、通常の実行環境では意識する必要はない。

実装を変更した際はまずここを通すこと。

3 パラメータの変更方法

すべてのパラメータは pyproject.toml の `[tool.flwr.app.config]` セクションで定義されている。恒久的に変えたい場合はこのファイルを直接編集し、一時的に変えたい場合は `--run-config` で上書きする（クイックスタート参照）。

3.1 基本の学習設定

キー	既定値	説明
`num-server-rounds`	5	通信ラウンド数
`fraction-evaluate`	0.2	各ラウンドで評価に参加させるクライアントの割合
`local-epochs`	1	クライアント 1 台あたりのローカル学習エポック数
`learning-rate`	0.001	学習率（AdamW）
`batch-size`	25	ミニバッチサイズ

3.2 Robust Aggregation 関連

キー	既定値	説明
`aggregation-method`	"fedavg"	集約手法。"fedavg" / "median" / "trimmed_mean" / "krum" / "bulyan"
`num-malicious-assumed`	-1	防御側が想定する悪性クライアント数の上限 f。-1 は「未指定」を意味し、実際に生成された悪性クライアント数がそのまま使われる。0 は「悪性クライアントがいないと防御側が想定する」という**有効な設定**であり、未指定とは区別されるため、番兵値として -1 を用いている。0 以上の値を明示すると、実際の悪性クライアント数と切り離して「防御側の想定」だけを変えられる（過小/過大評価の実験に使える）

Krum/Bulyan の成立条件

Krum は n ≥ f+2、Bulyan は n ≥ 4f+3（n=参加クライアント数、f=num-malicious-assumed）を満たさないと計算できない。特に Bulyan は悪性割合が高い設定（例：num-clients=100 かつ malicious-ratio=0.3）では n ≥ 4f+3 を満たせず、意図的に ValueError が送出される。これはバグではなく、Bulyan というアルゴリズム自体の既知の限界であり、run_experiments.py はこれを検知して failures.csv に記録し次の条件へ進む。

3.3 データ分布（IID / Non-IID）

キー	既定値	説明
`data-split`	"non_iid"	"iid"：全クライアントへ均等分配。"non_iid"（"dirichlet" でも可）：Dirichlet 分布で偏りを導入
`dirichlet-alpha`	0.6	Non-IID 時の偏りの強さ。値が小さいほど偏りが強い（0.1 で激偏り、10 でほぼ IID 相当）
`seed`	42	データ分割の再現性用シード。malicious-seed とは独立に管理される。同一条件を複数回（seed 違いで）繰り返す際にこれを変える

3.4 悪性クライアント生成

キー	既定値	説明
`num-clients`	100	クライアント総数。federation 側の `num-supernodes` と必ず一致させること（§8 参照）
`malicious-ratio`	0.1	悪性クライアントの割合（0.0 / 0.1 / 0.2 / 0.3 を想定）。人数は `round(num-clients × malicious-ratio)` で計算される
`malicious-seed`	42	malicious-ratio から悪性クライアント ID を自動生成する際のシード（seed とは独立）
`malicious-ids`	""	手動で ID を指定したい場合はここに "0,1,5" のようにカンマ区切りで書く。空でなければこちらが優先され、malicious-ratio は無視される

3.5 攻撃設定

キー	既定値	説明
`attack-type`	"none"	"none" / "sign_flip" / "scale" / "adaptive"（§5 参照）
`poison-rate`	0.2	将来の label-flip 等の攻撃で使う想定のパラメータ（現状の sign_flip/scale/adaptive では未使用）

3.6 出力先

キー	既定値	説明
`output-dir`	"."	`final_model.pt` と `fixed_eval/` の保存先。run_experiments.py が実験条件ごとに別ディレクトリ（`experiments/<条件名>/`）を自動的に渡すことで、実験間の上書きを防いでいる

4 Robust Aggregation の仕組み

`aggregation.py` の `aggregate()` 関数が集約手法を切り替えるディスパッチャになっている。

- **FedAvg**：データ数による重み付き平均（唯一、重み付けを行う手法）
- **Median**：各パラメータ座標ごとの中央値
- **Trimmed Mean**：各座標について上下 beta 個（既定では f と同数）を除外してから平均
- **Krum**：他クライアントとの距離が最も近い 1 台の更新をそのまま採用（平均は取らない）
- **Bulyan**：Krum の反復選択で $\theta = n - 2f$ 台を選び、その後 Trimmed Mean で集約

集約はサーバ側の `pytorchexample/robust_strategy.py`（RobustFedAvg クラス）内、`aggregate_train()` で行われる。各クライアントから届いたモデル重み（state_dict）を 1 次元ベクトルに変換し、`aggregate()` に渡してから元の state_dict へ復元している。

5 攻撃の仕組み

5.1 sign_flip / scale（更新量ベース）

悪性クライアントが**クライアント側**で適用する単純な攻撃。学習後、

```
Δ = w_local − w_global
```

を計算し、sign_flip なら −Δ、scale なら 10Δ として、`w_global + Δ_malicious` をサーバへ送信する（`client_app.py` の `train()` 内）。

5.2 adaptive（Adaptive Model Poisoning Attack）

Fang ら [1] の「集約手法の特性を突く」という考え方をベースにした独自実装。この攻撃は他クライアントの更新や使用中の集約手法を知っているという前提（論文でいう full-knowledge 攻撃者）が必要だが、Flower の各クライアントプロセスは独立しているため、クライアント側ではこの前提を満たせない。そこで**サーバ側**（`robust_strategy.py` の `aggregate_train()`）で、全クライアントの更新を受け取った直後・集約する直前に、悪性クライアント分のベクトルをクラフトした攻撃済みベクトルへ差し替える設計にしている。

- **Median / Trimmed Mean 向け**：honest な更新の座標ごとの最大値・最小値を基準に、攻撃方向へ「範囲外」の値を割り当てる（悪性クライアント間で押し出し具合をずらし、trimming で丸ごと除外されるのを防ぐ）
- **Krum / Bulyan 向け**：悪性クライアントを honest クラスタの中心付近に密集させ、「Krum に選ばれる最大の攻撃強度 λ」を二分探索で求める
- **FedAvg**：Adaptive 攻撃の対象外（元々 robust でないため sign_flip/scale で十分効く）

> **原論文との対応について**
>
> 本実装は論文の中核アイデアを踏まえた近似・簡略化した再実装であり、論文本文の数式をそのまま再現したものではない。卒論で原論文との厳密な数式対応を主張する場合は、`attack.py` の docstring に記載した設計方針と原論文の該当箇所を突き合わせて確認すること。

6 run_experiments.py の使い方

実験計画書 5.2 の比較条件（集約手法 5 種 × 悪性割合 4 種 × データ分布 2 種 × 3 回繰り返し = 120 条件）を自動で回すスクリプト。

```
python3 run_experiments.py        # 全120条件を実行（未完了分のみ）
python3 run_experiments.py --quick  # 動作確認用の小規模設定（4条件、10クライアント、1ラウンド）
```

6.1 仕組み

- スクリプト冒頭の `AGGREGATION_METHODS` / `MALICIOUS_RATIOS` / `DATA_SPLITS` / `SEEDS` / `NUM_CLIENTS` / `NUM_SERVER_ROUNDS` を変更すると条件を調整できる（`NUM_SERVER_ROUNDS` は仮値なので、予備実験で決めた値に差し替えること）
- 各条件は `experiments/<条件名>/` へ出力される（条件名の形式：`<集約手法>_<データ分布>_ratio<悪性割合>_seed<seed>`）
- `attack-type` は全条件で "adaptive" に固定している（実験計画書 5.2 の比較条件に攻撃種別は含まれておらず、本研究の主眼が Adaptive 攻撃の評価であるため）

6.2 完了判定と resume（再開）

`experiments/<条件名>/final_model.pt` の有無で完了済みかどうかを判定する。すでに存在する条件はスキップされるため、スクリプトを中断して再実行すれば未完了分だけ再開できる。

メモ

`flwr run` はサブプロセスとして例外発生時でも終了コード 0 を返すことがある（Flower 内部でエラーがログに出るだけでプロセス自体は正常終了扱いになるため）。そのため成功判定を終了コードではなく `final_model.pt` の実在で行っている。

6.3 失敗の記録

成功しなかった条件は `experiments/failures.csv` に記録され、スイープ全体は止めずに次へ進む。列は `timestamp, condition, command, output_tail`（実行時刻・条件名・実行コマンド・出力の末尾 30 行）。Bulyan の成立条件を満たさないケースなどがここに記録される想定である。

7 ログ・出力ファイルの読み方

7.1 logs/results.csv（ラウンドごとの結果）

`server_app.py` の `global_evaluate()` から毎ラウンド 1 行追記される。

列	内容
`run_id`	1 回の実行を一意に識別する文字列（集約手法_データ分布_ratio_seed_実行時刻）
`round`	通信ラウンド番号（0 はグローバルモデル初期化直後の評価）
`accuracy / loss_func`	そのラウンド時点のテストデータに対する精度・損失
`aggregation_method`	使用した集約手法
`data_split`	iid / non_iid
`malicious_ratio`	悪性クライアント割合
`attack_type`	使用した攻撃種別
`seed`	データ分割の再現性用シード

7.2 logs/summary.csv（実行 1 回につき 1 行）

学習終了後（`strategy.start()` 完了後）に 1 行だけ追記される。results.csv とほぼ同じ列構成だが、`round / accuracy / loss_func` の代わりに `final_round / final_accuracy / final_loss`（最終ラウンドの値）を持つ。実験全体の比較（visualize_results.py）はこのファイルを主に使う。

7.3 logs/plots/（visualize_results.py の出力）

```
python3 visualize_results.py
```

results.csv/summary.csv を読み込み、複数 seed の平均値・標準偏差を用いて以下 6 種類の PNG を `logs/plots/` に生成する。

ファイル名	内容
`accuracy_transition.png`	ラウンドごとの Accuracy 推移（悪性割合 × データ分布でパネル分割、集約手法ごとに色分け）
`loss_transition.png`	同上、Loss 版
`final_accuracy_bar.png`	最終 Accuracy の棒グラフ（集約手法 × 悪性割合、データ分布でパネル分割）
`final_loss_bar.png`	同上、Loss 版
`iid_vs_noniid.png`	IID/Non-IID 比較図（悪性割合ごとにパネル分割）
`aggregation_robustness.png`	集約手法ごとの頑健性曲線。悪性割合を横軸に最終 Accuracy をプロットし、攻撃が強くなるほど各集約手法がどれだけ性能低下するかを示す。本研究の結論を最も端的に示す図になる想定

7.4 その他の個人ログ（笠原さん用、提出物ではない）

`log_output.py` 冒頭に明記の通り、以下は課題評価とは無関係な個人的な実行ログである。

- `logs/client_metrics.csv` ：クライアントごとのラウンド別 train_loss/eval_loss/eval_acc
- `logs/server_metrics.csv` ：サーバ側のラウンド別 accuracy/loss_func（条件タグなし、単発実行の確認用）
- `logs/run_config.txt` ：実行のたびに追記される簡易な設定ログ
- `experiments/<条件名>/fixed_eval/round_XXX/` ：固定した 32 枚の画像に対する予測結果（preds.csv・grid.png・誤分類画像 mis/*.png）。学習の進み具合を目視確認するためのもの

8 よくある注意点・トラブルシューティング

- **flwr run に `--stream` を付け忘れると即座に返ってしまう**：`--stream` なしだと実行を投入するだけで完了を待たずにコマンドが終了する。ログを見ながら完了を待ちたい場合や、スクリプトから完了を検知したい場合は必ず `--stream` を付けること（run_experiments.py は内部で自動的に付けている）。
- **`num-clients` と federation の `num-supernodes` は必ず一致させる**：`num-clients` は悪性クライアント ID を生成する範囲を決めるだけで、実際にシミュレーションで起動するクライアント数は federation 設定（既定 10、`flwr federation simulation-config` や `--federation-config 'num-supernodes=N'` で変更）が決める。ずれると「存在しないはずのクライアントが悪性指定される」等の不整合が起きる。

```
flwr run . local-simulation --stream \
  --run-config 'num-clients=100' \
  --federation-config 'num-supernodes=100'
```

- **`num-malicious-assumed` の既定値は -1（未指定の意味）**：0 を指定すると「防御側は悪性クライアントがいないと想定する」という設定になり、Krum/Trimmed Mean/Bulyan の防御が実質無効化される（この2つを混同すると、集約手法を変えても性能が変わらないように見えるバグ的な挙動になるので注意）。
- **Bulyan は悪性割合が高いと意図的に失敗する**：num-clients=100 で malicious-ratio=0.3（f=30）の場合、$n \geq 4f+3 = 123$ を満たせないため ValueError で失敗する。これは Bulyan というアルゴリズムの既知の限界であり、run_experiments.py 経由なら failures.csv に自動記録される。
- **長時間セッションで flwr run がハングすることがある**：多数の実行を繰り返すと、蓄積したバックグラウンドプロセス（Ray/SuperLink のワーカー）が原因で処理が進まなくなることがある。CPU 使用率がほぼ 0% のまま長時間止まっている場合は、該当プロセスを終了して flwr をやり直すか、ローカルの SuperLink（ `~/.flwr/local-superlink` ）を再起動すると解消することが多い。

9 実験計画書との対応表

項目	内容	対応ファイル
7.1	MNIST + DNN 版 task.py	`pytorchexample/task.py`
7.2	Robust Aggregation	`aggregation.py`，`pytorchexample/robust_strategy.py`
7.3	悪性クライアント生成	`malicious.py`
7.4	攻撃実装	`attack.py`，`pytorchexample/client_app.py`
7.5	CSV 保存	`pytorchexample/log_output.py`，`pytorchexample/server_app.py`
7.6	run_experiments.py	`run_experiments.py`
7.7	可視化	`visualize_results.py`

参考文献

[1] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, Neil Zhenqiang Gong, "Local Model Poisoning Attacks to Byzantine-Robust Federated Learning," Proceedings of the 29th USENIX Security Symposium (USENIX Security 2020), pp. 1605–1622, 2020.